

Lecture 15

*Lecturer: Prof Minati De**Scribe: Team 15***Scribed by:**

1. Abhineet Singh (2022MT11736)
2. Ishant Yadav (2022MT11397)
3. Saksham Saldhi (2022MT61299)
4. Samyak Jain (2022MT11658)
5. Subhojit Ghosal (2022MT61976)

1 Overview

In Lecture 14, we studied the **Pumping Lemma for Context-Free Languages (CFLs)** — its formal statement, intuition via parse trees, and how the pumping length is determined. We then applied it to prove that certain languages (e.g., $a^n b^n c^n$, $\{ww \mid w \in \{0,1\}^*\}$, and factorial-length strings) are **not context-free**. Finally, the topic of **Turing Machines** was introduced.

In this lecture, we introduce the **Turing Machine** as a general and most powerful automaton model, capable of simulating any computation. We define its formal structure, study its operation with examples, and show how it can accept languages beyond the context-free class.

2 From Finite Automata to Turing Machines

Finite automata (DFA/NFA) could only process input strings in a single left-to-right scan. They had no additional memory and could *only read*, never modify, the input. This limitation meant that once the head moved forward, all past information was lost.

Pushdown automata extended this model by adding a stack as storage. However, even pushdown automata are limited in power: some simple languages such as $\{a^n b^n c^n\}$ and $\{ww \mid w \in \Sigma^*\}$ cannot be recognized. To go beyond this, we consider a more flexible model of computation: the **Turing Machine**.

Key Features of a Turing Machine

A Turing machine can be viewed as an automaton with a semi-infinite tape and a read-write head. The tape serves both as input and working memory. With this setup, several new capabilities emerge compared to DFA/NFA:

1. **Two-way head movement:** Unlike NFA/DFA where the head only moved right, the Turing machine head can move both left and right. This allows revisiting earlier parts of the input instead of “forgetting” past information.
2. **Read and Write operations:** In DFA/NFA the input was fixed and could only be read. In contrast, the Turing machine tape can be both read and modified at any time.
3. **Semi-Infinite tape:** The tape is an unbounded one-dimensional array of cells, providing unlimited storage. This is a crucial jump in computational power.
4. **Explicit accept and reject states:** Finite automata only had accept states. A Turing machine halts in either an *accept* or a *reject* state, making the outcome explicit.

Although the mechanism is simple, the Turing machine model is powerful enough to capture any algorithmic process. This universality underpins the **Church–Turing thesis**: any computation that can be performed mechanically can be carried out by a Turing machine.

3 Mathematical Definition of Turing Machine

A Turing Machine is a formal automaton that extends the power of DFA and PDA by allowing a read–write tape of infinite length. Formally, it is defined as a **7-tuple**:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$$

where

1. Q is a finite set of states.
2. Σ is the input alphabet, not containing the blank symbol $\sqcup$.
3. Γ is the tape alphabet, with $\Sigma \subseteq \Gamma$ and $\sqcup \in \Gamma$.
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function. Given the current state and current tape symbol, it outputs the new state, the symbol to be written, and the head movement (Left or Right).
5. $q_0 \in Q$ is the start state.
6. $q_{\text{rej}} \in Q$ is the reject state.
7. $q_{\text{acc}} \in Q$ is the accept state.

A Turing machine may be seen as a simple computer. Its control has a finite memory (the states), while the tape provides unbounded storage. The transition function δ acts as the “program” of the machine: at each step, the machine reads the current symbol, possibly overwrites it, changes state, and moves the head left or right. Computation halts when the machine enters an accepting or rejecting state, or when δ is undefined.

Configuration / Instantaneous Description (ID)

An **Instantaneous Description (ID)**, or configuration, is a complete snapshot of a Turing Machine at one point in time. It is formally represented by a string that includes the current state, the tape contents, and the head position.

Formal Notation The standard notation for an ID is:

$$a_1 a_2 \dots a_{k-1} q a_k a_{k+1} \dots a_n$$

Where:

- $q \in Q$ is the current state of the machine.
- The string $a_1 a_2 \dots a_n$ represents the non-blank portion of the tape.
- The read/write head is positioned on the symbol immediately to the right of q , which is a_k .

Example: Initial Configuration The **initial configuration** is the ID when the machine begins its computation. The machine is in its start state, q_0 , and the head is positioned on the leftmost symbol of the input string w .

For an input string $w = abba$, the initial configuration is:

$$q_0 abba$$

4 Language Accepted by a Turing Machine

A string is considered **accepted** by a Turing Machine if, starting from the initial state, there exists a sequence of transitions that leads the machine to enter a final (or accept) state. Upon entering an accept state, the machine halts.

The set of all strings that a Turing Machine M accepts is called the **language of the machine**, denoted $L(M)$.

A language is called **Turing-recognizable** if there exists a Turing Machine that accepts every string in that language.

5 Drawbacks of Turing machine

The definition of language acceptance only specifies the outcome for strings that are in the language. For a string w that is not in $L(M)$, one of two outcomes is possible:

1. The machine may halt in a non-final state.
2. The machine may enter an infinite loop and never halt.

Any string for which the machine does not halt is, by definition, not in $L(M)$. This possibility of non-termination for certain inputs is a fundamental property of Turing machines.

Definition: A Turing Machine (TM) is called a **decider** if it halts on every possible input string. A decider never enters an infinite loop.

For any given input string w , a decider is guaranteed to finish its computation in one of two ways:

- If the string is in the machine's language ($w \in L(M)$), the decider will halt in an **accept state**.
- If the string is not in the machine's language ($w \notin L(M)$), the decider will halt in a **reject state**.

A language for which a decider exists is called a **decidable language**.

Example 1

We are interested in designing a Turing Machine to accept the language:

$$L = \{ww \mid w \in \{0, 1\}^*\}.$$

To simplify the design, we instead construct a machine for:

$$L' = \{w\#w \mid w \in \{0, 1\}^*\},$$

where the separator “#” helps in matching the two halves of the input.

- The machine scans the input, marking symbols from the first half and verifying them against the second half.
- If all symbols match around the “#” and the tape is exhausted correctly, the machine halts in q_{acc} .
- Otherwise, if a mismatch occurs, it halts in q_{rej} .

Thus, the Turing Machine provides the necessary computational power beyond PDA to handle languages like ww .

Sequence of Moves

We demonstrate the working of a Turing Machine that accepts

$$L' = \{w\#w \mid w \in \{0, 1\}^*\}.$$

Step 1: Initial Configuration

0	0	1	#	0	0	1	
---	---	---	---	---	---	---	--

Head is on the first symbol. Mark it as read (replace by X) and move the head right.

Step 2: After Marking First Symbol

X	0	1	#	0	0	1	
---	---	---	---	---	---	---	--

Continue moving the head right until the symbol ' $\#$ ' is seen. Then cross over and find the first unmarked symbol on the right of ' $\#$ '.

Step 3: Matching Symbols

X	0	1	#	X	0	1	
---	---	---	---	---	---	---	--

Match the unmarked symbol after ' $\#$ ' with the corresponding marked one on the left. If they match, mark it as X .

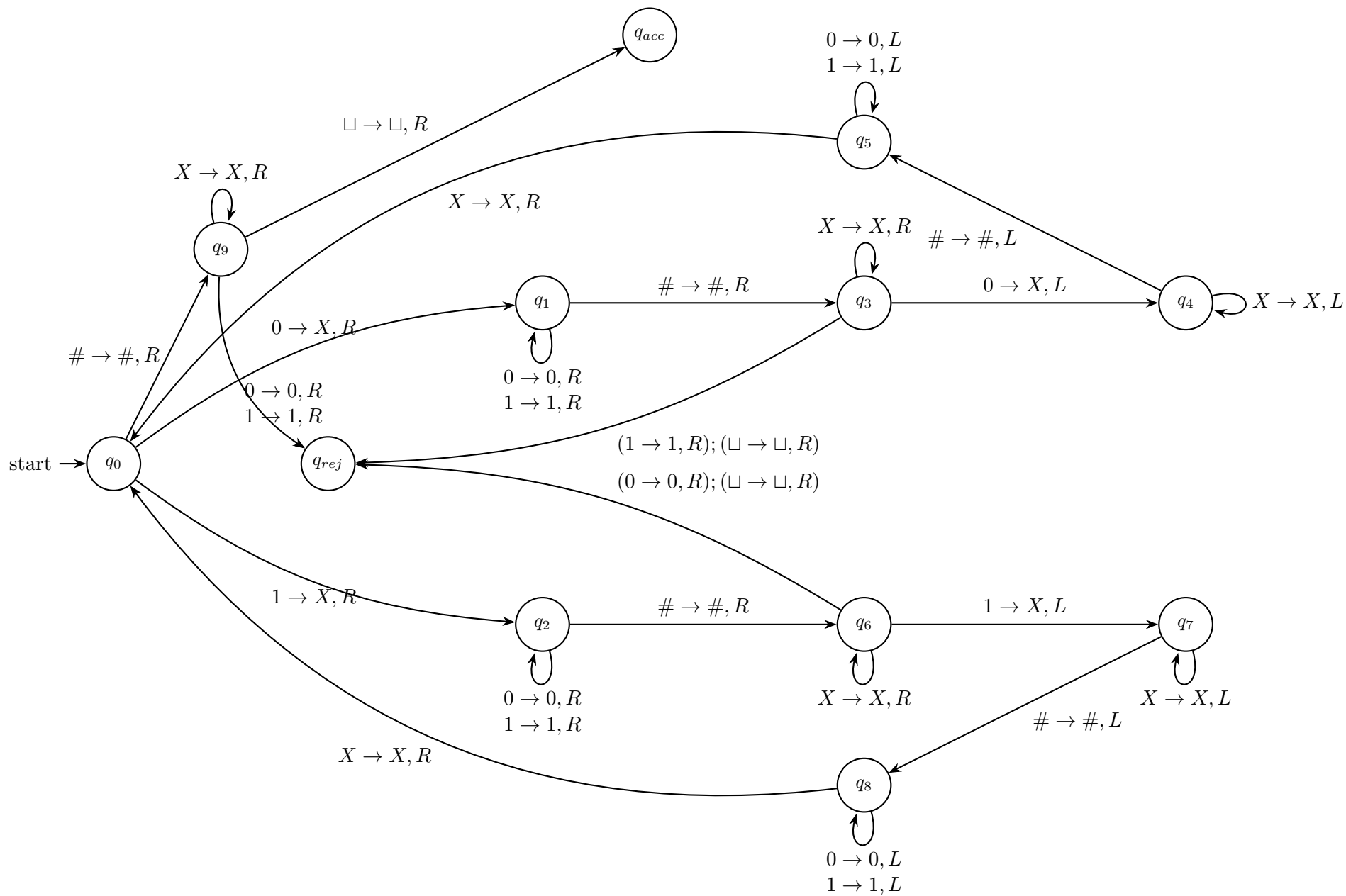
Step 4: Repeat Process

X	X	X	#	X	X	X	
---	---	---	---	---	---	---	--

Now move the head left until an ' $\#$ ' and X is seen (X should be seen after ' $\#$ '). The head now points on a X . Move the head to the right by one cell to read the next unread symbol and then repeat the entire process (until all symbols are marked). If every pair matches, go to the accept state q_{acc} . Otherwise, reject.

State Diagram

The state diagram for the turing machine is shown on the next page.



Example 2 (Homework)

Designing a Turing Machine to accept the language:

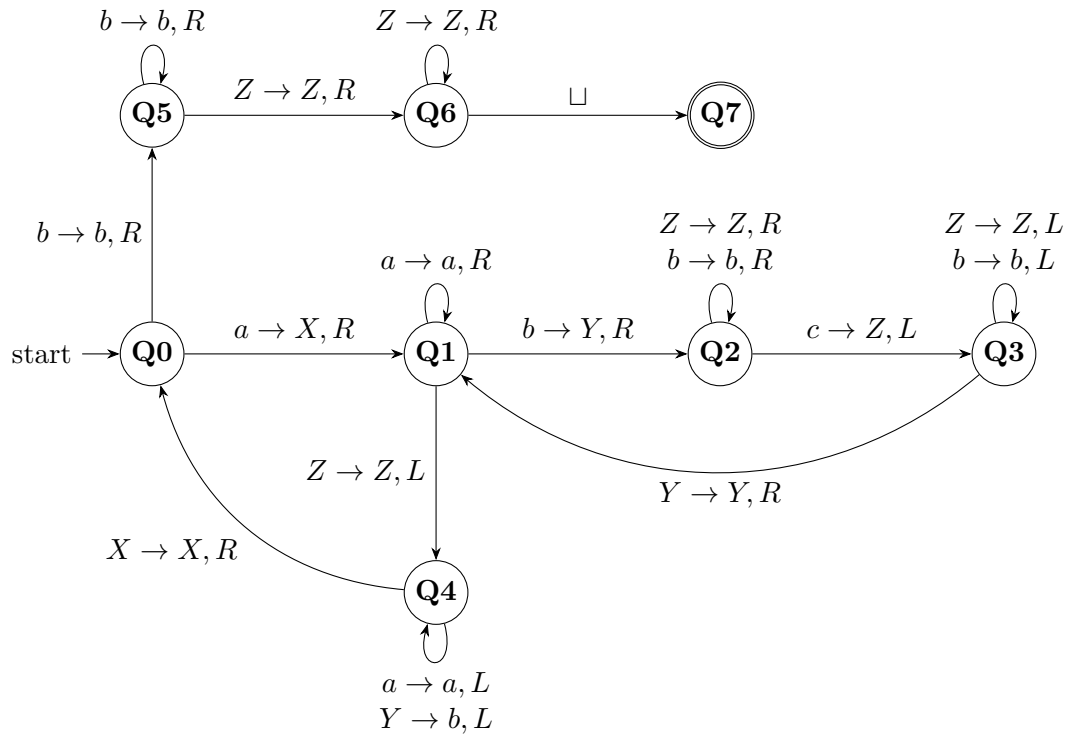
$$L = \{a^i b^j c^k \mid i, j, k \geq 1 \text{ and } k = i \cdot j\}.$$

Intuition and Strategy

The Turing Machine verifies that the input is of the form $a^i b^j c^k$ with $i, j, k \geq 0$. It then checks that the number of c 's equals $i \times j$ by matching each a with all b 's, and for each b , matching one c . The Turing Machine proceeds by marking the symbols sequentially to ensure all c 's are matched exactly.

1. Start scanning the tape from the leftmost symbol.
2. Otherwise, search for the first unmarked a .
 - If no unmarked a is found, go to Step 9.
3. Mark this a as X to show it is processed.
4. From here, find the first unmarked b .
 - If no unmarked b is found, go to Step 8.
5. Mark this b as Y to show it is processed.
6. For this b , find an unmarked c and mark it as Z .
 - If no unmarked c is found when needed, reject the string.
7. Repeat marking c 's for all b 's corresponding to the current a .
8. After all b 's for the current a are processed, restore all Y 's back to b 's and return to find the next unmarked a (Step 4).
9. When no unmarked a 's remain:
 - Check if all c 's are marked.
 - If yes and no unprocessed b 's remain, accept the input.
 - Otherwise, reject the input.

State Diagram for Turing Machine



For any state, all input symbols for which there are no outgoing transitions from that state are implicitly directed to the reject state, for simplicity of the diagram they are assumed as convention.

Explanation of the State Diagram

The transitions are in the form:

read symbol $\rightarrow$ write symbol , head move

where R means move Right, and L means move Left.

1. Q0 (Start State):

- Scans for the first unmarked **a** from the left.
- On reading an **a**, marks it as **X** and moves to Q1 to process this **a**.
- If it reads a **b**, all **a**'s are processed; moves to Q5 to check for unmatched **b**'s.

2. Q1 (Marking a 'b'):

- After marking an **a**, finds the first unmarked **b**.
- On reading **b**, marks it as **Y** (linking to the marked **a**) and moves to Q2.

3. Q2 (Marking a 'c'):

- For each (X,Y) pair, finds the next unmarked **c** and marks it as **Z**.
- If it finds a **c**, marks it as **Z** and moves left to Q3.

4. Q3 (Return Left):

- Moves left to return toward the last marked **a**.
- Locates the next unmarked **a** to start the process again for another product cycle.

5. Q4 (Cleanup and Restart):

- Moves left across any symbols, returning to Q_0 when the tape is at the beginning.
- Looks for more unmarked **a**'s or to finish if all are marked.

6. **Q5 (Checking Completion of 'b' Processing):**

- Handles the case when all **a**'s are marked.
- Checks that all **b**'s are also processed; if so, moves to Q_6 for cleanup, else rejects.

7. **Q6 (Verification of 'Z's):**

- Checks that all **c**'s have been used (now marked as **Z**).
- On reading **Z**, continues right. If it finds the blank symbol, moves to the accepting state Q_7 .

8. **Q7 (Accept State):**

- The final state: if tape scanning is completed successfully (all markers used correctly and all **c**'s marked), the input string is accepted as part of the language.

Example 3 (Homework)

Designing a Turing Machine to accept the language:

$$L = \{a^{2^n} \mid n \geq 0\}$$

The core principle is to **repeatedly divide the number of a by two**. The input is a power of two if and only if this division can be repeated until the number of a becomes exactly one.

Execution Steps:

1. **Sweep and Mark:** The TM performs a pass from left-to-right across the tape. For every two a it encounters, it crosses out the second one (e.g., by writing an X over it). This effectively halves the number of a in one pass.
2. **Check for Odd Numbers:** If at the end of a pass, there was an “unpaired” a left over, the count was odd. Since any power of two (except for 1) must be even, an odd count greater than one means the original number was not a power of two. The Turing Machine will **reject**.
3. **Rewind and Repeat:** If the pass was successful (the count was even), the TM rewinds its head to the start of the tape and repeats Step 1 on the remaining, unmarked **a**'s.
4. **Final Check and Acceptance:** The process repeats until one of two things happens:
 - If a pass starts and there is only **one** a left on the tape, the original string's length was a power of two. The Turing Machine halts and **accepts**.
 - If a pass results in an odd count (as in Step 2), the Turing Machine **rejects**. An empty input is also rejected.

Formal Definition of the Turing Machine:

- **States Q :** $\{Q_0, Q_1, Q_2, Q_3, Q_4, Q_{acc}, Q_{rej}\}$
- **Input Alphabet Σ :** $\{a\}$
- **Tape Alphabet Γ :** $\{a, x, \sqcup\}$ (where ' $\sqcup$ ' is the blank symbol)
- **Start State:** Q_0
- **Final (Accept) State F :** $\{Q_{acc}\}$
- **Reject State:** Q_{rej}

State discription:

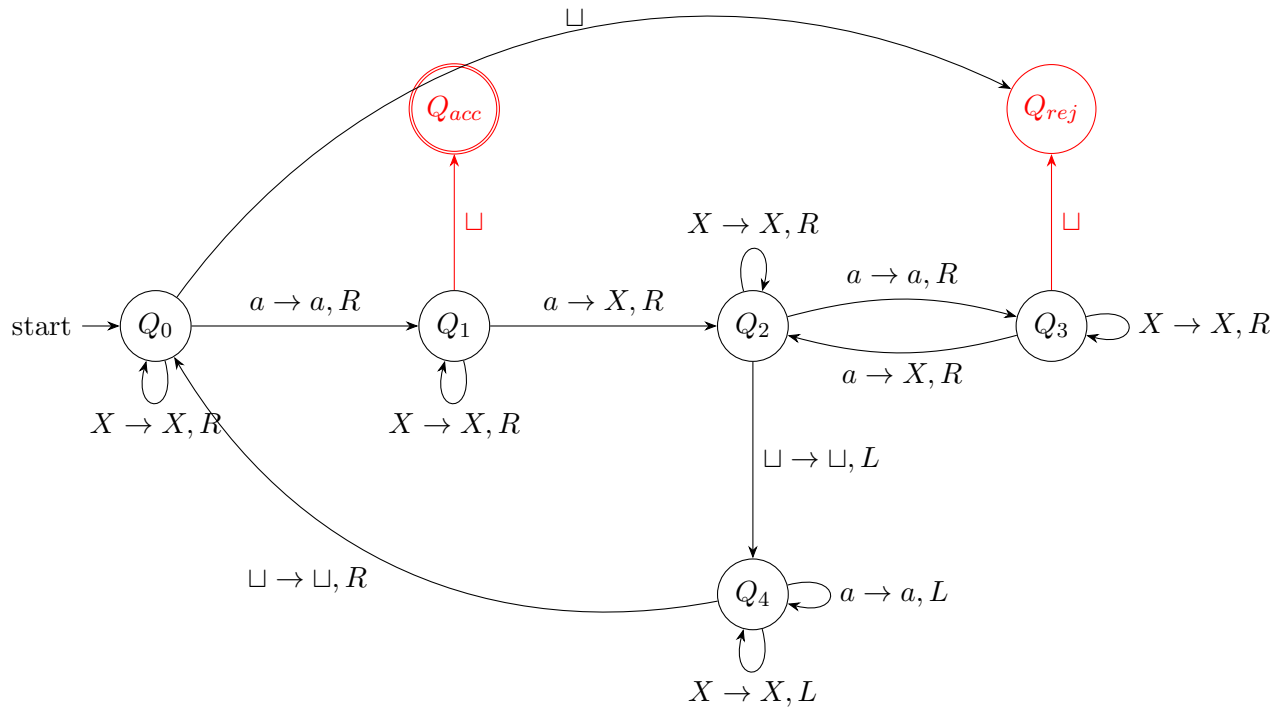
Q_0 : The initial state. It scans right to find the first 'a'. It also handles the empty string case.

Q_1 : **Seen One 'a'**. This state checks if the 'a' found by Q_0 is the only one left on the tape. If so, it accepts. Otherwise, it begins the marking process.

Q_2 : **Even Parity Mark**. The machine enters this state after successfully finding a pair of 'a's and marking the second one. The count of processed 'a's in the current pass is even. It looks for the next 'a' to keep.

Q_3 : **Odd Parity Mark**. The machine enters this state after finding the first 'a' of a new pair. The count of processed 'a's is odd. It looks for the second 'a' to mark. If it encounters Blank symbol then rejects the string (as number of a encountered is odd and greater than 1)

Q_4 : **Rewind**. After a successful pass (an even number of 'a's were found and halved), this state rewinds the tape head to the beginning to start a new pass from Q_0 .



References

- [1] P. Linz, *An Introduction to Formal Languages and Automata*, 6th Edition, Jones & Bartlett Learning, 2016.